# Using Codex for Product/Harness Engineering

Sharing research and workflows
from 6 months of heavy Codex usage

# Hi, I'm Danial Hasan

I'm an applied AI engineer building products and harnesses with agents.

# I'm playing with Codex to build better agentic products.

Agentic products combine two types of engineering: the product surface and the agent harness. Codex can drive both.

## Product engineering

Build the product surface users interact with.

## Harness engineering

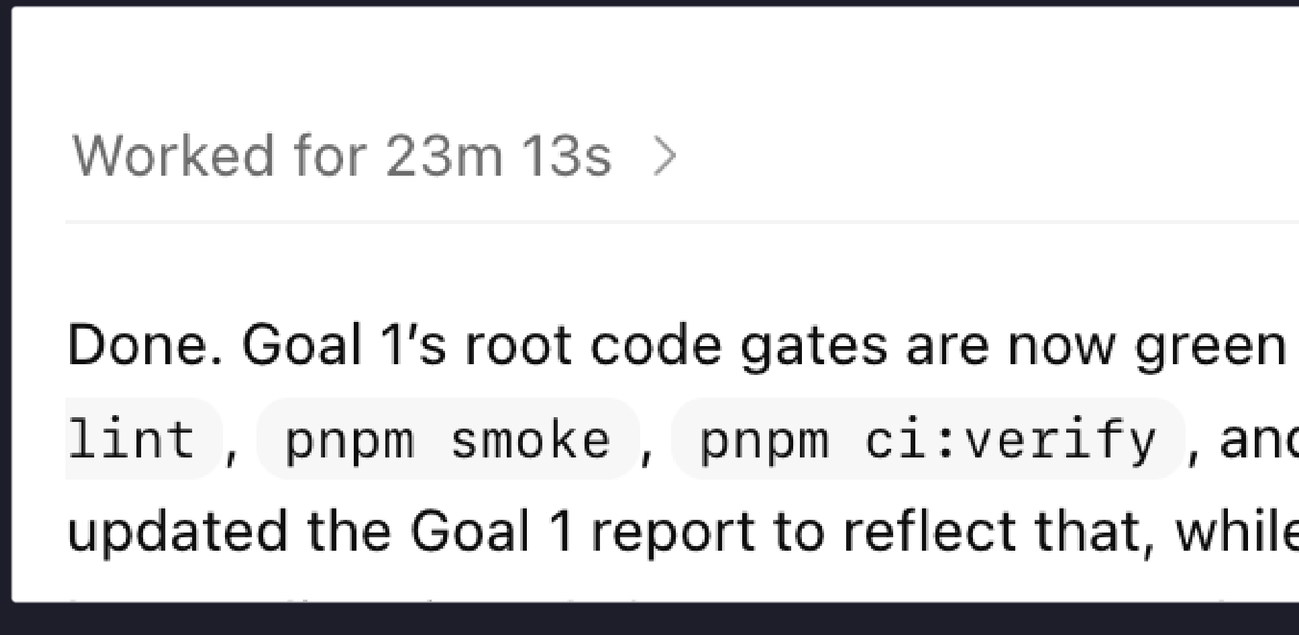
Shape the agent system to complete tasks we care about.

# Codex for Product Engineering

I started by using Codex the obvious way: brain dump into it, refine that into a plan, then let it execute.

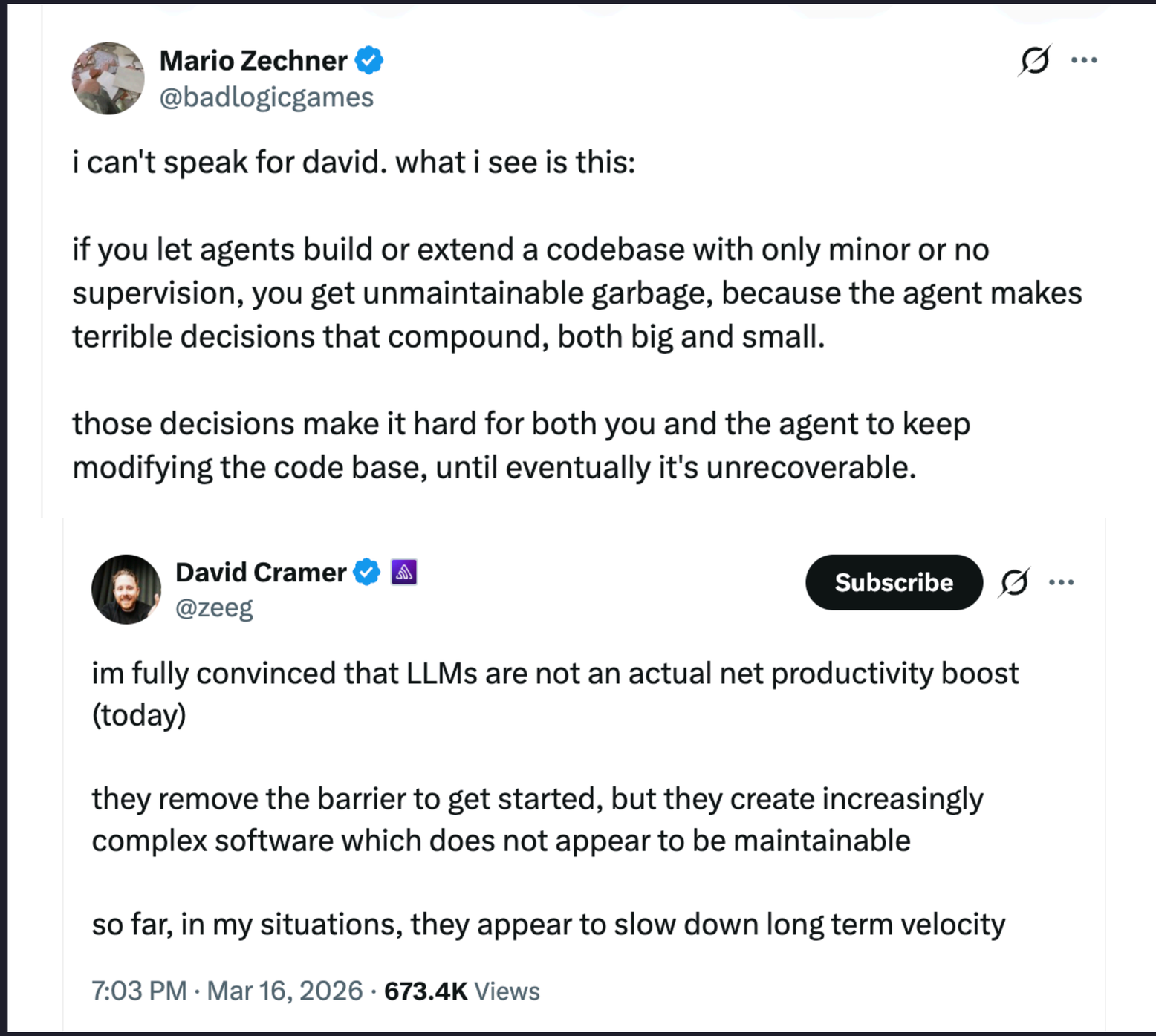
# It worked surprisingly well.

Codex could move real code and get visible product changes onto the screen.



# Vibe coding doesn't scale cleanly.

- It finished pieces, but missed the whole job.
- It built parts that were not wired together.
- I still had to figure out what happened.



**Mario Zechner**
@badlogicgames

i can't speak for david. what i see is this:

if you let agents build or extend a codebase with only minor or no supervision, you get unmaintainable garbage, because the agent makes terrible decisions that compound, both big and small.

those decisions make it hard for both you and the agent to keep modifying the code base, until eventually it's unrecoverable.

**David Cramer**
@zeeg

Subscribe

im fully convinced that LLMs are not an actual net productivity boost (today)

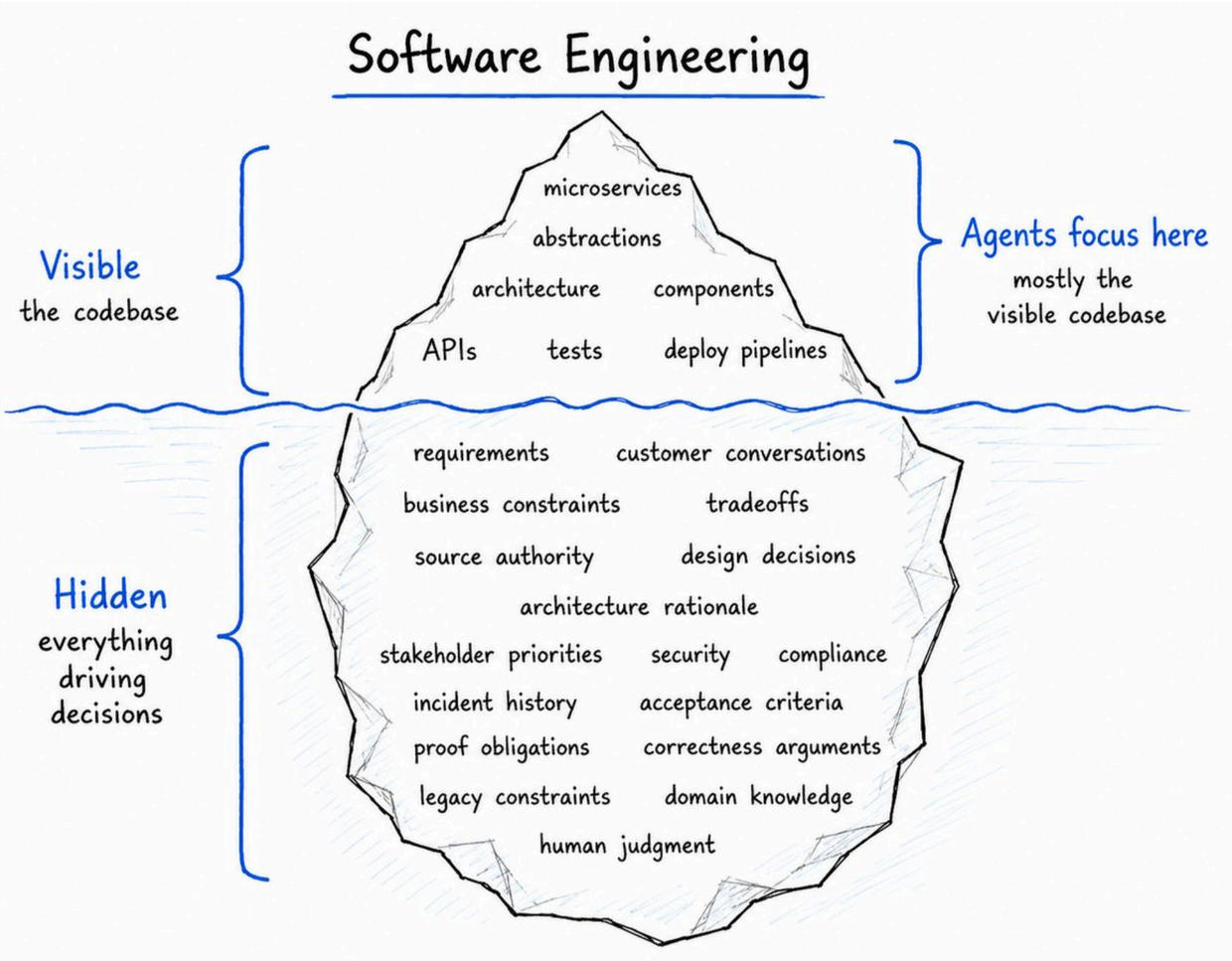
they remove the barrier to get started, but they create increasingly complex software which does not appear to be maintainable

so far, in my situations, they appear to slow down long term velocity

7:03 PM · Mar 16, 2026 · **673.4K** Views

# Why do agents like Codex fail in this context?

- Code shows what changed, but not why it mattered.
- Agents can edit files without knowing the real decision.
- When proof is missing, review becomes detective work.



# So how do we fix it?

We look for what engineers in the 90s came up with, encode those into skills, and serve them in the harness.

# We turn old engineering discipline into skills Codex can run.

| | |
|---|---|
| requirements engineering | Turn messy intent into claims we can verify. |
| implementation contracts | Tell Codex what to change and what not to touch. |
| execution orchestration | Split work into lanes agents can own. |
| layered verification | Match each claim to the proof that actually checks it. |
| receipt closeout | Leave links a human or agent can audit later. |

# Plan mode is too naive when requirements are recursive.

Requirements engineering solves these problems:

- One prompt hides multiple actors, scenarios, and business contexts.
- Plans jump to implementation before asking what must be true.
- Source authority, fit criteria, and risky assumptions stay implicit.
- Proof gets substituted: screenshots for state, backend tests for journeys.
- Review becomes reconstructing intent from a plausible final diff.

# Requirements engineering makes plans better.

The extra context gives Codex something concrete to verify against.

### actors and scenarios

Who it matters to, the business context, and the scenario they are in.

### distinct claims

Each user, system, state, permission, and review claim is split apart.

### source authority

Which code, doc, ticket, customer source, or external source is allowed to decide.

### fit criteria and proof

What must be true, and which proof surface can actually validate it.

### forbidden substitutions

Auth flow: screenshot is not enough.
Need traces and DB verification.

### verification and judgment

What the agent verifies autonomously, and what the human still has to judge.

# Without requirements engineering, Codex writes a plausible plan.

The plan makes sense, but it waves away too much detail, and that's how agents produce garbage.

I'll implement this by first finding the relevant files, then updating the UI and backend logic to support the new behavior. After that I'll run the existing tests and do a quick manual check in the browser.

Plan:

1. Find the component or route that owns this flow.
2. Update the UI copy and controls.
3. Add any missing backend support.
4. Wire the frontend to the backend.
5. Run lint, typecheck, and tests.
6. Open the app locally and confirm the feature works.

I'll keep the change scoped and avoid touching unrelated code.

# With requirements engineering, Codex writes a verifiable map.

The extra detail makes it easy to catch issues before they happen, and makes agentic work easier to verify. This means you can delegate more ambitious work with Codex.

Before planning implementation, I'm going to turn the request into requirements we can verify.

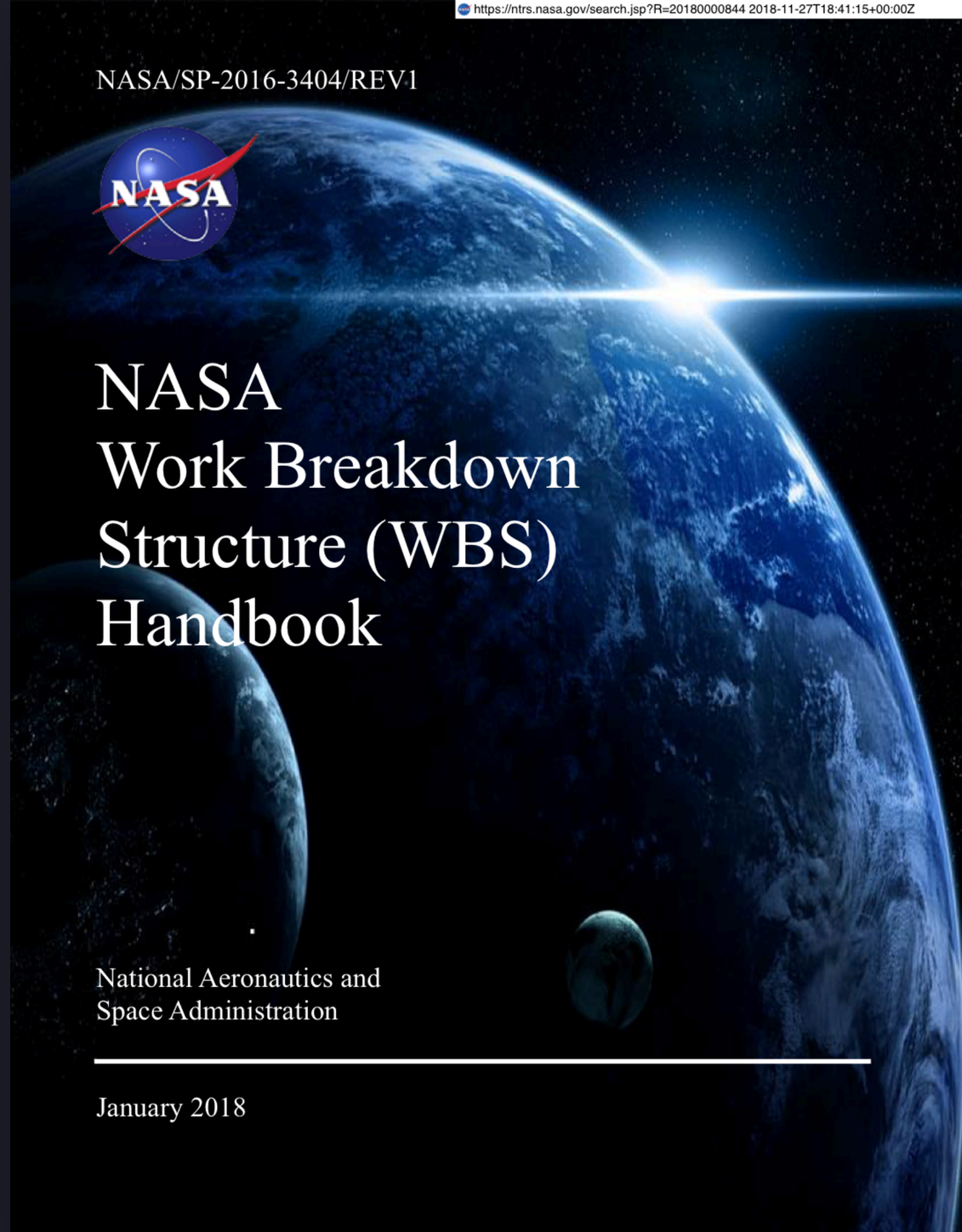
Requirements map:

| Claim | Actor | Fit criterion | Proof surface |
|---|---|---|---|
| The user can complete the target flow without ambiguity. | End user | The happy path is visible and executable in the app. | Browser walkthrough + screenshot |
| The system preserves the right state after the action. | Product/system | Refreshing or reopening shows the expected persisted state. | DB/query proof or durable read |
| The UI explains what happened without leaking internal machinery. | Reviewer/user | Copy names the user-facing outcome, not implementation details. | Screenshot review |
| Failures are visible and recoverable. | User/operator | Error state explains the blocker and next action. | Negative-path test |
| Review can happen from evidence, not reconstruction. | Human reviewer | The final packet links requirement, change, and proof. | Receipt / trace / test output |

# Agents must be taught how to orchestrate their workers.

Execution orchestration solves these problems:

- Agents collide when no one owns the lane.
- Tests break when another lane changes the contract.
- The human gets stuck merging everything by hand.



# Execution orchestration makes delegation safe.

It turns approved work into owned, isolated, verifiable lanes.

### lane ownership

Which worker owns which product seam, file surface, and proof obligation.

### dependency waves

What can run together, and what must wait for another lane to land.

### serialized resources

Shared runtimes, DB state, Electron slots, and deploy surfaces that cannot collide.

### worktree isolation

Dedicated branch and worktree lanes for mutating parallel work.

### proof responsibility

Each lane knows what evidence it must leave behind before review.

### review convergence

How isolated work collapses back into one human-judgable packet.

# Without execution orchestration, Codex creates busy workers.

The work splits, but ownership does not. That is how agents collide.

I tried to merge the worker outputs, but the agents touched the same code.

Conflict repair:

| Problem | What happened | Now I have to do |
|---|---|---|
| Same component changed twice | Worker A changed the interaction surface while Worker B changed the same component for state wiring. | Manually reconcile the component boundary. |
| Contract drift | One worker renamed the API field while another wrote tests against the old name. | Decide which contract is real and update both sides. |
| Runtime collision | Two workers assumed they could use the same app/runtime proof path. | Re-run verification in a serialized order. |
| Review confusion | Each worker left partial proof for its own local change. | Rebuild one coherent review packet. |

# With execution orchestration, Codex delegates owned work.

Parallelism becomes reviewable because every lane has a boundary, proof target, and convergence path.

Execution map:

| Wave | Lane | Ownership | Isolation | Proof |
|---|---|---|---|---|
| A | UI lane | interaction copy and visible state | dedicated worktree | browser receipt |
| A | State lane | persistence and query contract | dedicated worktree, shared component serialized | DB/query proof |
| B | Runtime proof lane | browser + DB verification | serialized runtime slot | trace + screenshots + query output |
| C | Review lane | evidence packet and reconciliation | integration worktree | human QA packet |

# With execution orchestration, Codex delegates owned work.

Parallelism becomes reviewable because every lane has a boundary, proof target, and convergence path.

| Work item | Depends on | Shared surface | Collision risk |
| --- | --- | --- | --- |
| UI interaction update | approved contract | `MissionReviewPanel` | overlaps with state wiring |
| State/query wiring | persistence contract | mission query model | overlaps with UI props |
| Browser verification | UI + state lanes complete | local dev server | must run after both lanes |
| DB verification | state lane complete | local DB / seeded data | serialized with browser proof |
| Review packet | all proof lanes complete | receipts and trace links | must converge last |

# Implementation starts when `/goal` executes a signed-off Linear contract.

By this point, Codex is not discovering what to build from scratch. It is executing a contract that already contains the seam, lanes, and proof obligations.

`/goal`

### execution entrypoint

The user points Codex at the implementation ticket, not a loose prompt.

### Linear implementation contract

Requirements engineering and execution orchestration are already compiled into the ticket.

### dev/test composition

Codex enters the multi-agent implementation, review, patch, and verification loop named by the contract.

# The implementation contract embeds the orchestration and proof plan.

The ticket is structured so Codex can spend compute without expanding scope.

### seam

The one product surface, backend boundary, or runtime behavior being changed.

### non-goals

Explicit surfaces, ideas, and shortcuts Codex must not touch.

### source of truth

Canon, current runtime state, contract rows, DB/API surfaces, and allowed local state.

### lane map

Worker ownership, worktrees, file surfaces, dependencies, and serialized resources.

### dev/test composition

Multi-agent implementation, non-author review, patch loop, and re-review path.

### verification matrix

The required proof layers and receipt targets before closeout can be trusted.

# Implementation is an iterative dev, review, and patch loop.

The code change is only one step inside the contract-owned loop.

### read contract

Load the seam, proof matrix, lane ownership, and forbidden scope.

### inspect code

Find the existing primitives, state owners, commands, and tests.

### implement

Make the smallest green change inside the owned surface.

### review + patch

Non-author review finds drift, bugs, missing proof, and abstraction mistakes.

### receipt

Leave evidence, non-claims, and any remaining HOLD for the next gate.

# Layered verification lets you trade compute for robust engineering.

Every layer is considered, then run, marked not required, or held with a reason.

### repo floor
lint, typecheck, test, build

### property
invariants and generated cases

### unit
local behavior and edge cases

### contract
API, schema, command, or component contract

### model
prompt, eval, trajectory, or model behavior proof

### integration
cross-module and runtime interaction proof

### smoke
the smallest live path that should not break

### DB-backed
persisted state, RLS, query, or projection proof

### Electron / UI
renderer, route, click path, and visible green state

### Computer Use
human-like journey proof in the real app surface

### telemetry
runtime queries that distinguish happy and negative paths

### receipt
one verdict: VERIFIED, PARTIAL, or UNVERIFIED

# Pick the verification layer for the part of the app you changed.

Different parts of the app need different verification layers.

| | |
|---|---|
| backend contract | API, contract, integration, and endpoint proof. |
| durable state | DB-backed write, projection, RLS, query, or telemetry proof. |
| UI journey | Electron route proof, click path, screenshots, and visible state. |
| human interaction | Computer Use walkthrough or human runtime QA packet. |
| model behavior | Eval, trace, trajectory, or dataset-backed repair proof. |

# Closeout tells the human exactly what to review.

The packet links each result to the proof or blocker behind it.

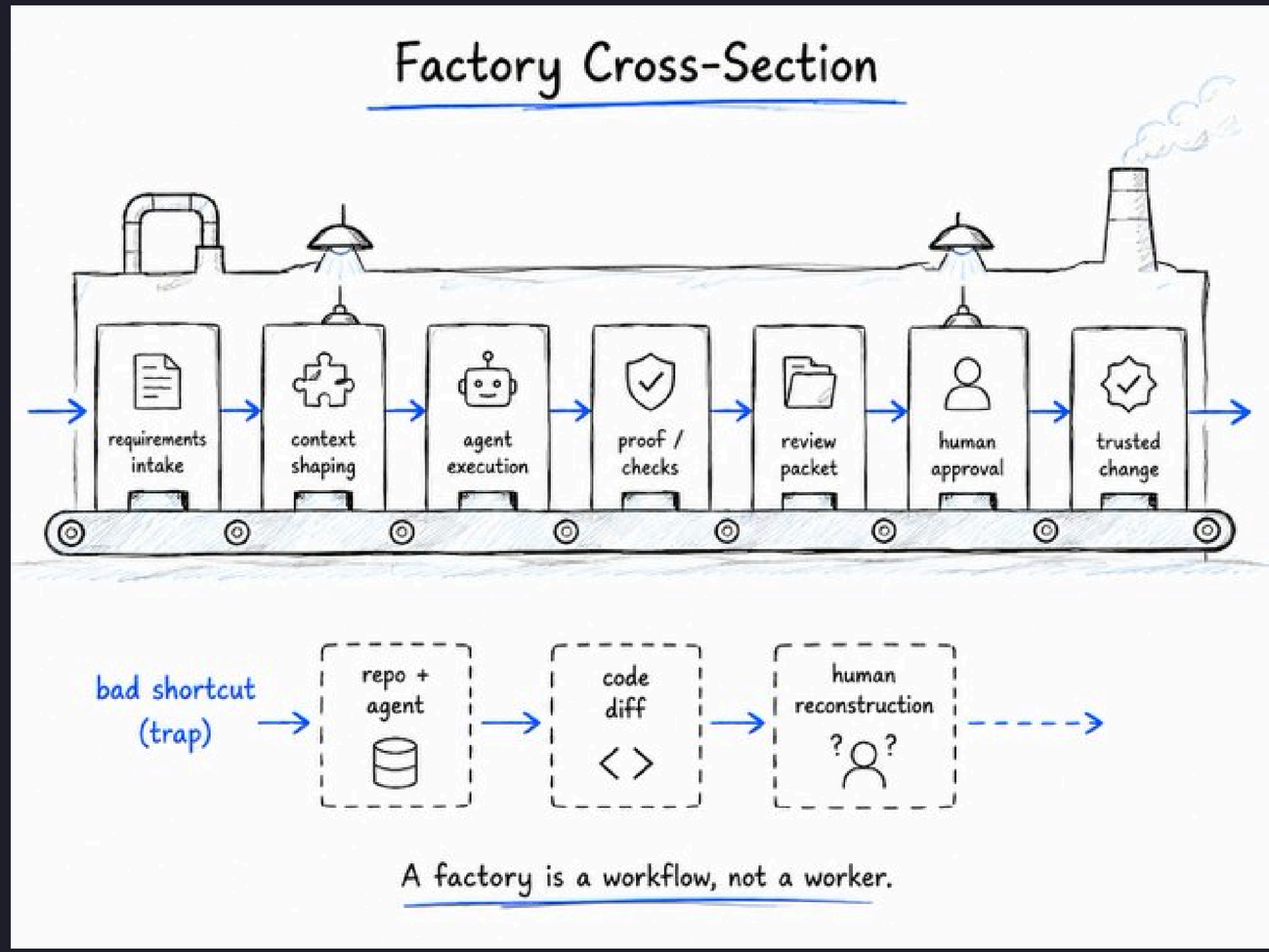
- Start with a short review index.
- Show what passed, what is blocked, and what was not claimed.
- Link the exact receipts the human should inspect.
- Keep proof gaps visible beside the evidence.

# These are early signs of a software factory.

Codex can move the code.

The hard part is keeping the context.

A factory is the workflow around the agent: requirements, proof, review, and trusted change.



Links

# Three links to take with you.

Everything is a markdown file nowadays smh.

## More research

Scan for a detailed breakdown on what powers software factories.

## Workflow skills

Scan for the plugin that contains the skills discussed in the presentation.

## Check out the product